



Object-Oriented Programming in PHP

OOP concepts in PHP from classes and objects to inheritance, encapsulation, and real-world projects. Designed for students stepping into professional programming.

PHP FUNDAMENTALS

Introduction to PHP OOP

Procedural vs. Object-Oriented Programming

Procedural programming organizes code as a sequence of function calls and steps - like

**Procedural
(Old Style)**

**OOP
(Modern Style)**

a recipe. **Object-Oriented Programming (OOP)** organizes code around objects that bundle data and behavior together, making large projects easier to manage, reuse, and scale.

Code runs top to bottom. Functions are called in sequence. Data and logic are separate.

Code is organized into objects. Each object holds its own data and actions. Easier to scale.

 PHP 5 and later fully support OOP. Modern PHP frameworks like Laravel and Symfony are built entirely on OOP principles.

Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects** rather than functions and logic. An object represents a real-world entity - like a

Student, a Car, or a BankAccount - and bundles together its **properties** (data) and **methods** (actions it can perform).



The Car Analogy

A **Car** has properties: color, brand, speed. It has methods: start(), brake(), accelerate(). The blueprint is the **class**; each car on the road is an **object**.



The Student Analogy

A **Student** has properties: name, ID, course. It has methods: enroll(), study(), submitAssignment(). Every enrolled student is a unique object.



The Bank Account Analogy

A **BankAccount** has properties: accountNumber, balance. It has methods: deposit(), withdraw(), checkBalance(). Each account holder gets their own object.

Reusability

Write once, use many times across your project

Maintainability

Easier to find and fix bugs in organized code

Scalability

Build large systems without chaos

Real-World Modeling

Code mirrors how we think about the world

Classes and Objects

Key Definitions

A **class** is a blueprint or template that defines what properties and methods an object will have. An **object** is a specific instance created from that class - like a cookie cutter (class) and the actual cookie (object).

- **Class:** The blueprint - defines structure and behavior
- **Object:** The instance - a real entity built from the class
- **\$this:** Refers to the current object inside a class
- **new:** Keyword used to create a new object

Syntax and Example

```
<?php
class Student {
    // Properties (attributes)
    public $name;
    public $course;

    // Method (behavior)
    function introduce() { echo
"My name is " . $this->name . "
and I study " . $this->course;
    }
}

// Creating objects
$student1 = new Student();
$student1->name = "Kelvin";
$student1->course = "IT";
```

```
$student1->introduce();
```

```
$student2 = new Student();
```

```
$student2->name = "Aisha";
```

```
$student2->course = "Business";
```

```
$student2->introduce();
```

```
?>
```

❏ The arrow operator `->` is used to access properties and methods of an object. Inside the class, `$this->` refers to the current object's own properties.

Properties, Methods, Constructors & Destructors

Properties are variables that store an object's data (also called attributes). **Methods** are functions defined inside a class that describe what an object can do. Together, they define the complete behavior and state of an object.

Properties and Methods in Action

```
<?php
class Student {
    // Properties
    public $name;
    public $age;
    public $grade;

    // Method
    public function
study($subject) {
    echo $this->name . " is
studying " . $subject;
}

    // Method with return value
    public function getGrade()
{
    return $this->grade;
}
```

Constructors and Destructors

A **constructor** (`__construct()`) runs automatically when an object is created. It is used to initialize properties. A **destructor** (`__destruct()`) runs when the object is destroyed, useful for cleanup tasks like closing database connections.

```
<?php
class Student {
    public $name;
    public $id;

    // Constructor
    public function
__construct($name, $id) {
    $this->name = $name;
    $this->id = $id;
    echo "Student " . $this->name
. " created.<br>";
}
```

```
}

$obj = new Student();
$obj->name = "Kelvin";
$obj->age = 19;
$obj->grade = "A";
$obj->study("PHP");
echo $obj->getGrade(); //
Output: A
?>
```

```
}

// Destructor
public function __destruct() {
    echo "Student " . $this->name
    . " destroyed.<br>";
}
}

$s1 = new Student("Kelvin",
101);
$s2 = new Student("Aisha",
102);
// Destructors run
automatically at end of script
?>
```

Access Modifiers

Access modifiers control who can see and use a class's properties and methods. PHP provides three levels of visibility that help protect data and enforce good design.



public

Accessible from **anywhere**
- inside the class, outside the class, and in child classes. Use for methods meant to be called freely.



private

Accessible **only inside the same class**. Cannot be accessed from outside or child classes. Use to protect sensitive data like passwords.



protected

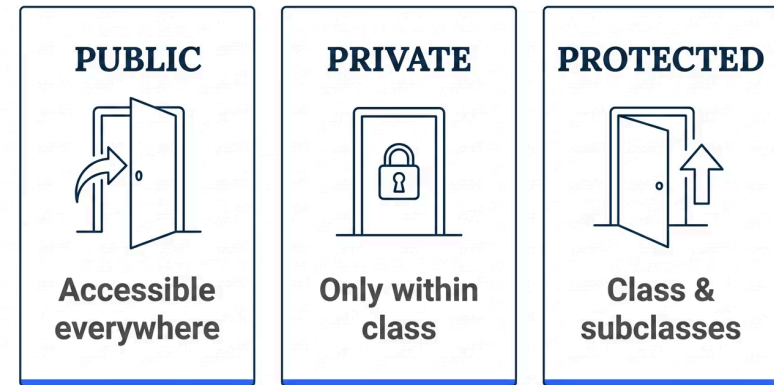
Accessible inside the **same class and its child classes**. Use when you want subclasses to inherit but still restrict outside access.

Demonstration

```
<?php
class Student {
    public $name;           //
    Anyone can access
    protected $grade;      //
    Class + children only
    private $password;     //
    This class only

    public function
    __construct($name, $grade,
    $pass) {
        $this->name = $name;
        $this->grade = $grade;
        $this->password =
    $pass;
    }

    public function showInfo()
    {
        echo "Name: " . $this-
    >name . "<br>";
    }
}
```



```
        echo "Grade: " . $this->grade . "<br>";
        // echo $this->password; // Works here
    }
}

$s = new Student("Kelvin", "A", "secret123");
echo $s->name;          // OK - public
// echo $s->grade; // ERROR - protected
// echo $s->password; // ERROR - private
$s->showInfo();
?~
```

Inheritance and Polymorphism

Inheritance

Inheritance allows a child class to inherit properties and methods from a parent class

Polymorphism

Polymorphism means "many forms." In OOP, it allows child classes to **override** methods from

using the `extends` keyword. This promotes **code reusability** - write common logic once in the parent, then extend or override it in children.

```
<?php
class Person {
    public $name;
    public $age;

    public function
__construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
    }

    public function greet() {
        return "Hello, I am " .
$this->name;
    }
}

// Child class inherits from
Person
class Student extends Person {
    public $course;
```

the parent class, providing their own specific implementation while sharing the same method name.

Method Overriding

A child class redefines a method it inherits. The child's version runs instead of the parent's when called on a child object.

Why It Matters

Each subclass can behave differently while sharing a common interface. Example: Dog and Cat both extend `Animal` and override `speak()`.

📄 **parent::** is used inside a child class to call the parent's version of a method or constructor, ensuring shared initialization logic still runs.

```
    public function
__construct($name, $age,
$course) {

parent::__construct($name,
$age); // Call parent
constructor
    $this->course =
$course;
    }

    // Override parent method
    public function greet() {
        return "Hi, I'm " .
$this->name .
        ", a " . $this->course . " student";
    }
}

$s = new Student("Kelvin", 19,
"IT");
echo $s->greet();
// Output: Hi, I'm Kelvin, a IT
```

```
student
```

```
?>
```

Encapsulation, Abstraction & Interfaces

Encapsulation

Encapsulation bundles data and methods together while **hiding internal details** from the outside world. Properties are declared **private** and accessed through **getter** and **setter** methods, giving you control over how data is read and modified.

```
<?php
class BankAccount {
    private $balance; // Hidden
    from outside
```

Abstraction

An **abstract class** cannot be instantiated directly. It defines a template that child classes must follow. Any class that **extends** an abstract class **must implement** all its abstract methods.

```
<?php
abstract class Shape {
    public $color;

    public function
    __construct($color) {
```

```

        public function
__construct($initial) {
            $this->balance =
$initial;
        }

        // Getter - read access
        public function
getBalance() {
            return $this->balance;
        }

        // Setter - controlled
write access
        public function
deposit($amount) {
            if ($amount > 0) {
                $this->balance +=
$amount;
            }
        }
    }

    $acct = new BankAccount(1000);
    $acct->deposit(500);

```

```

        $this->color = $color;
    }

    // Abstract method - must
be implemented by children
        abstract public function
area();

        public function describe()
{
            return "A " . $this-
>color . " shape";
        }
    }

    class Rectangle extends Shape {
        public $width;
        public $height;

        public function
__construct($color, $w, $h) {

parent::__construct($color);
            $this->width = $w;
            $this->height = $h;

```

```
echo $acct->getBalance(); //  
1500  
?>
```

```
    }  
  
    public function area() {  
        return $this->width *  
$this->height;  
    }  
}  
  
$r = new Rectangle("blue", 5,  
10);  
echo $r->area(); // 50  
?>
```

- ① **Interfaces** define a contract that classes must follow. A class uses `implements` to agree to provide all methods declared in the interface. Unlike abstract classes, interfaces cannot contain any implementation - only method signatures. A class can implement multiple interfaces.

Static Methods, Properties & Common Errors

Static Members

The **static** keyword allows properties and methods to belong to the **class itself** rather than to any individual object. Static members are shared across all instances and can be accessed without creating an object, using the `::` (scope resolution) operator.

```
<?php
class Student {
    public $name;
    public static $school =
    "Tech University";
    public static $count = 0;

    public function
```

Common Errors to Avoid

Missing \$this

Inside a class, always use `$this->name`, not just `$name`, to refer to the object's property.

Forgetting new Keyword

Writing `$s = Student()` instead of `$s = new Student()` causes a fatal error.

Visibility Errors

Abstract Class Instantiation


```

__construct($name) {
    $this->name = $name;
    self::$count++; //
Increment static counter
}

// Static method
public static function
getSchool() {
    return self::$school;
}

public static function
getTotalStudents() {
    return self::$count;
}
}

echo Student::getSchool();
// Tech University
$s1 = new Student("Kelvin");
$s2 = new Student("Aisha");
echo
Student::getTotalStudents(); //
2

```

Trying to access
a `private` or
`protected`
property from
outside the
class triggers
an error.

You cannot do
`new Shape()`
on an abstract
class. Only
concrete child
classes can be
instantiated.

?>

- ❏ Static methods cannot use `$this` because they are not tied to any specific object. Use `self::` to refer to the class itself.

Practice

STUDENT MANAGEMENT SYSTEM

This mini project demonstrates core OOP concepts in a practical school system. Study the code, then complete the exercises and quiz below.

```
<?php
// Abstract parent class
abstract class Person {
    protected $name;
```

```
protected $id;

public function __construct($name, $id) {
    $this->name = $name;
    $this->id = $id;
}

abstract public function getRole();
}

// Child class with inheritance
class Student extends Person {
    private $course;
    private $grades = [];
    public static $totalStudents = 0;

    public function __construct($name, $id, $course) {
        parent::__construct($name, $id);
        $this->course = $course;
        self::$totalStudents++;
    }

    // Encapsulation: setter
    public function addGrade($grade) {
        $this->grades[] = $grade;
    }
}
```

```

    }

    // Encapsulation: getter
    public function getAverage() {
        if (count($this->grades) == 0) return 0;
        return array_sum($this->grades) / count($this->grades);
    }

    public function getRole() {
        return "Student - " . $this->course;
    }

    public function introduce() {
        return "I am " . $this->name . ", ID: " . $this->id;
    }
}

// Usage
$s1 = new Student("Kelvin", 101, "IT");
$s1->addGrade(85);
$s1->addGrade(90);
echo $s1->introduce() . "<br>";
echo "Average: " . $s1->getAverage() . "<br>";
echo "Total students: " . Student::$totalStudents;

```

?>

Quick Quiz

1 What keyword creates an object from a class?

Answer: `new`

2 Which modifier hides a property from child classes?

Answer: `private`

3 What does `parent::__construct()` do?

Answer: Calls the parent class constructor

Practice Assignments



Exercise 1

Create a `Book` class with properties `title`, `author`, and `year`. Add a method `display()` that prints all details.



Exercise 2

Extend `Book` into a `Textbook` class that adds a `subject` property and overrides `display()`.



Exercise 3

Build a `Library` class that holds an array of `Book` objects and can add, remove, and list all books.

